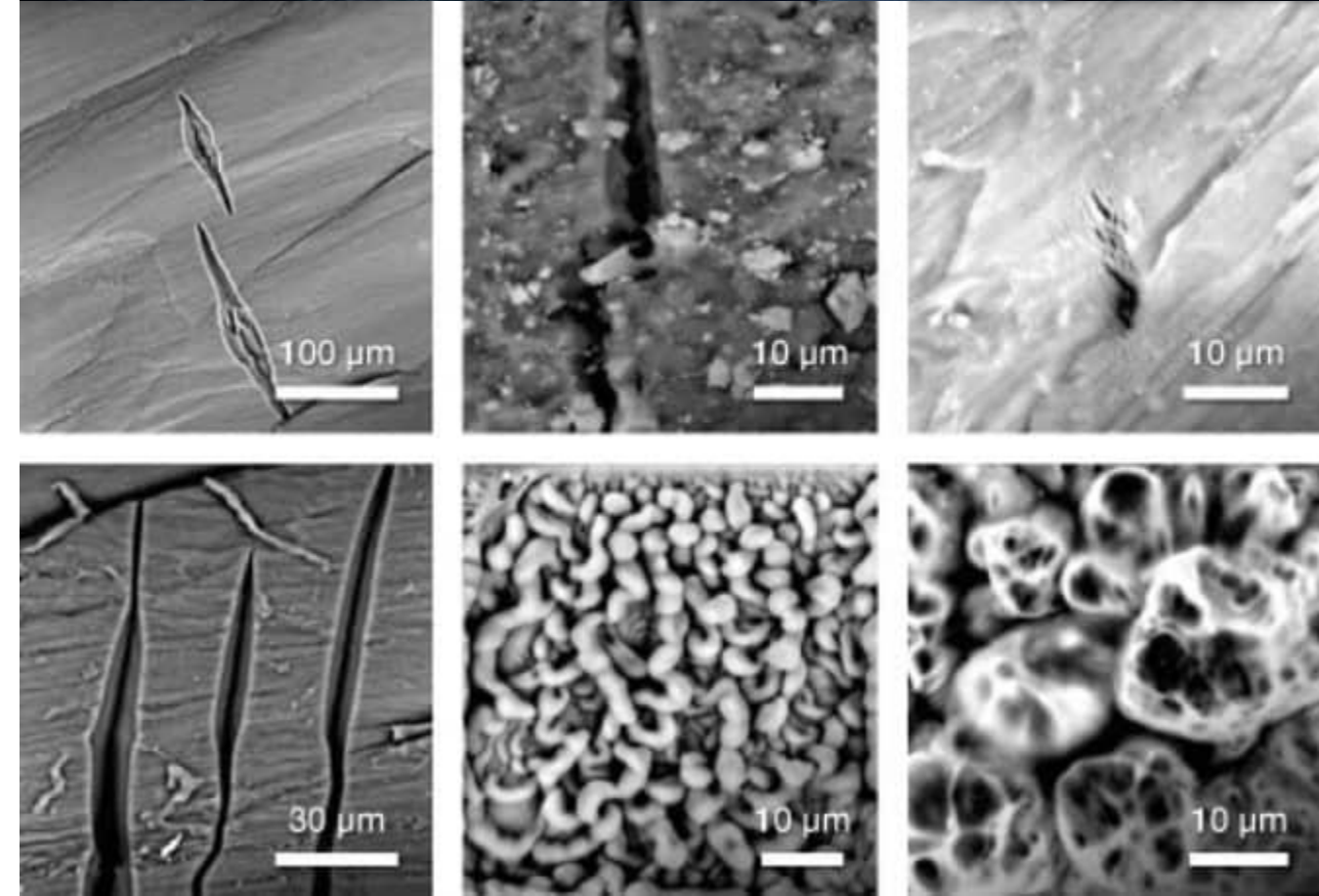


基於非監督式學習的 SEM 微結構缺陷偵測

Clustering + Autoencoder-based Anomaly Detection

作者：梁家毓 E54111213

國立成功大學 材料科學與工程學系



動機：

Why SEM ?

Why Unsupervised Learning?

- SEM 是材料科學中最重要的分析工具

人工判讀缺陷：

- 耗時
 - 主觀
 - 易漏判
- 非監督式學習
 - 缺陷通常非常稀少



Dataset and Pre-processing Strategy

數據來源

Kaggle GrainSize SEM

Dataset，提供不同晶粒尺寸的
微結構圖像。(422張影像)

<https://www.kaggle.com/datasets/dragonzhang/grain-size-train>

圖像前處理

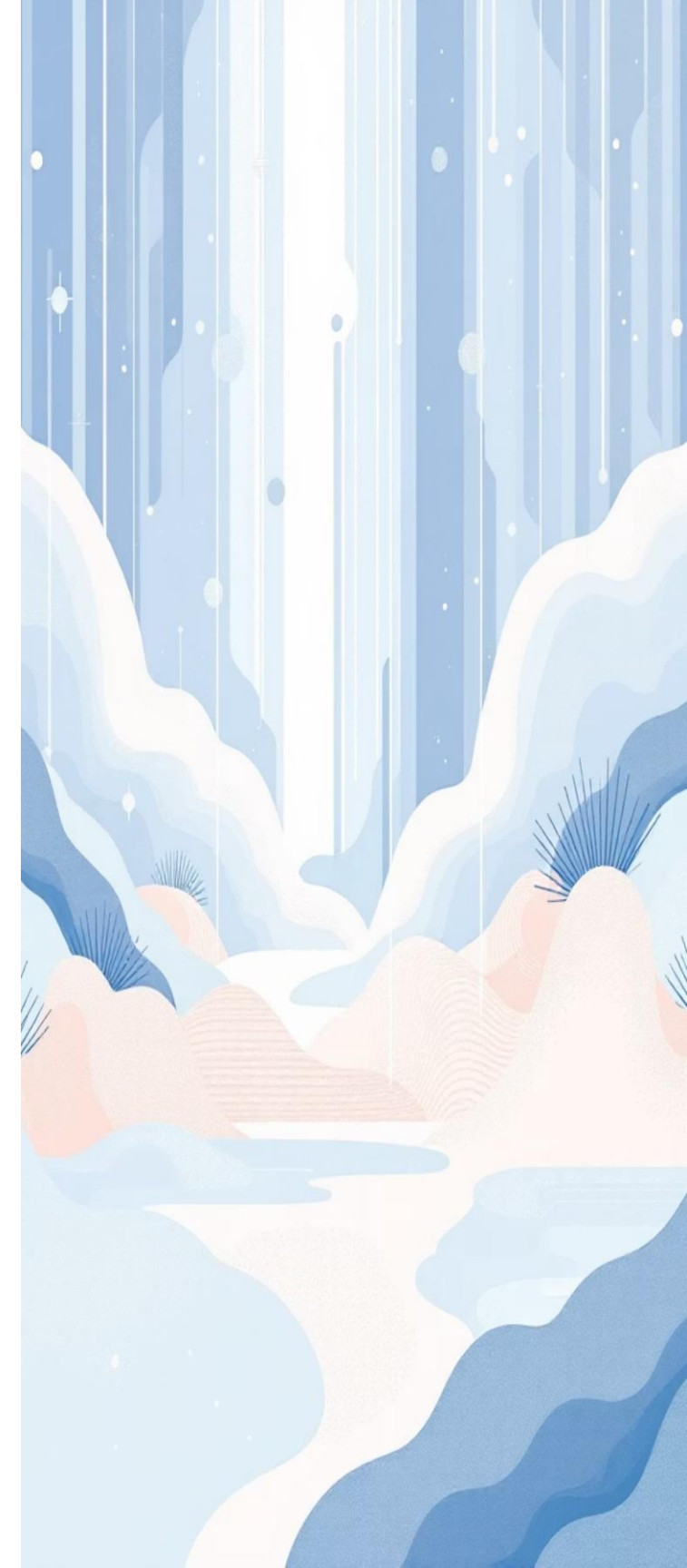
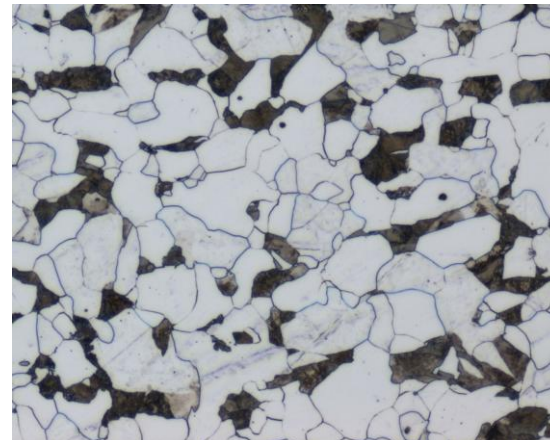
1. 將原始圖像 normalize
2. Resize (512*512)
3. 切割成超過 80,000 個
64x64 像素的小塊 (patches)

影像特性

高解析度灰階影像

有明顯的微結構紋理

沒有標註缺陷(非監督)



方法學流程

1

1. 圖像預處理（分塊）

將大型 SEM 圖像分割成小塊，增加數據量並聚焦局部特徵

2

2. 無監督式聚類

（主成分分析 PCA + K-Means），識別並分組相似的微結構模式

3

3. 模型訓練

（卷積自編碼器 CAE），學習正常微結構的低維表示

4

4. 缺陷檢測

（重建誤差分析），透過比較原始圖像與重建圖像的差異來定位缺陷



Dataset and Pre-processing Strategy (code)

```
def preprocess_image(path, resize_to=(512, 512)):
    img = cv2.imread(str(path), cv2.IMREAD_GRAYSCALE)
    if img is None:
        raise ValueError(f"Cannot read image: {path}")

    if resize_to is not None:
        img = cv2.resize(img, resize_to, interpolation=cv2.INTER_AREA)

    # 轉成 float32 然後做 normalize
    img = img.astype(np.float32) / 255.0

    return img

# 測試
img = preprocess_image(image_paths[0], resize_to=None)
print("shape:", img.shape, "min:", img.min(), "max:", img.max())
plt.imshow(img, cmap='gray')
plt.axis('off')
```

```
def extract_patches(img, patch_size=64, stride=32):
    H, W = img.shape
    patches = []
    coords = []

    for y in range(0, H - patch_size + 1, stride):
        for x in range(0, W - patch_size + 1, stride):
            patch = img[y:y+patch_size, x:x+patch_size]
            patches.append(patch)
            coords.append((y, x))

    patches = np.stack(patches, axis=0)
    coords = np.array(coords)
    return patches, coords

# 測試
patches, coords = extract_patches(img, patch_size=64, stride=32)
print("圖被切成幾個 patch :", patches.shape[0])
print("每個 patch 尺寸:", patches.shape[1:])

# 看幾個 patch 檢查
fig, axes = plt.subplots(1, 5, figsize=(12, 3))
for i, ax in enumerate(axes):
    ax.imshow(patches[i], cmap='gray')
    ax.axis('off')
plt.tight_layout()
plt.show()
```

圖被切成幾個 patch : 1386
每個 patch 尺寸: (64, 64)



Dataset and Pre-processing Strategy 2.(code)

```
PATCH_SIZE = 64
STRIDE = 32
MAX_PATCHES_TOTAL = 80000

all_patches = []
meta_info = []

total_patches = 0

for img_id, path in enumerate(image_paths):
    img = preprocess_image(path, resize_to=(512, 512))
    patches, coords = extract_patches(img, patch_size=PATCH_SIZE, stride=STRIDE)

    # 如果超過限制，就只取前面幾個
    if total_patches + patches.shape[0] > MAX_PATCHES_TOTAL:
        remain = MAX_PATCHES_TOTAL - total_patches
        if remain <= 0:
            break
        patches = patches[:remain]
        coords = coords[:remain]

    # 降成 float16 省一半記憶體
    patches = patches.astype(np.float16)

    all_patches.append(patches)
    img_ids = np.full((coords.shape[0], 1), img_id, dtype=np.int32)
    meta = np.hstack([img_ids, coords])
    meta_info.append(meta)

    total_patches += patches.shape[0]

    print(f"處理到第 {img_id} 張圖，累積 patch 數: {total_patches}")

    if total_patches >= MAX_PATCHES_TOTAL:
        break

all_patches = np.concatenate(all_patches, axis=0) # (N, 64, 64)
meta_info = np.concatenate(meta_info, axis=0) # (N, 3)

print("總共 patch 數量:", all_patches.shape[0])
print("meta_info 形狀:", meta_info.shape)

# 加 channel 維度
patches = all_patches[..., np.newaxis] # (N, 64, 64, 1)
meta = meta_info
print("patches 形狀:", patches.shape)
```

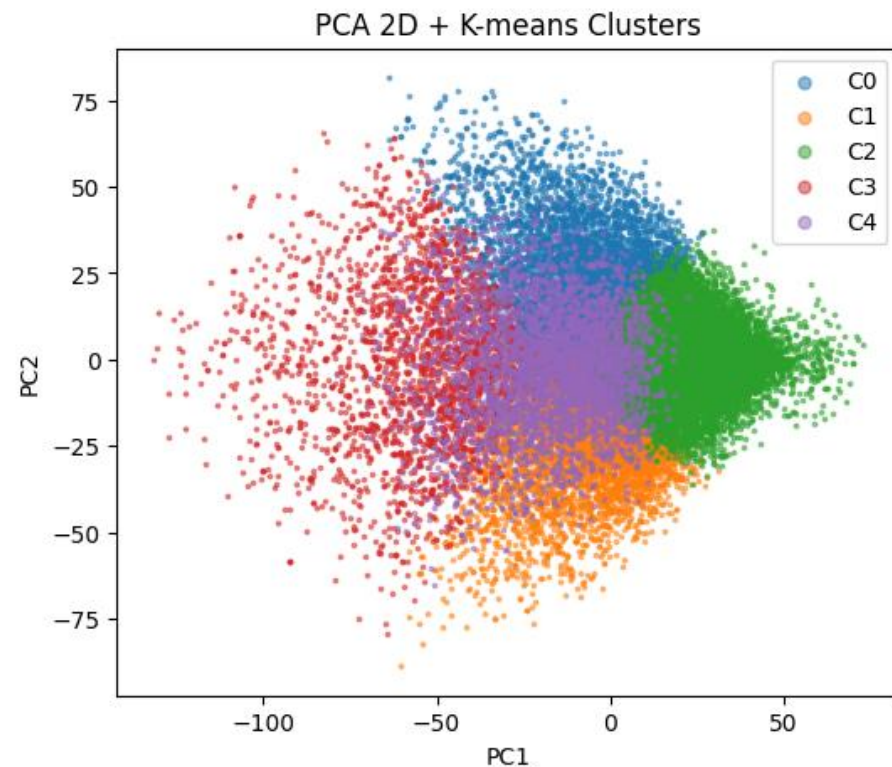
省記憶體以及減少訓練時間小秘訣

```
處理到第 354 張圖，累積 patch 數: 79875
處理到第 355 張圖，累積 patch 數: 80000
總共 patch 數量: 80000
meta_info 形狀: (80000, 3)
patches 形狀: (80000, 64, 64, 1)
```

微結構聚類與識別

PCA + K-means Clustering

- 主成分分析 (PCA): 用於降低數據維度，提取關鍵特徵。
4096(64*64) 維 → PCA 壓到 50 維
- K-Means 聚類: 將晶粒結構自動劃分為 5 個形態學上的類別
各群代表不同微結構 pattern
- Cluster 作圖: 把 PCA 降為到 2 維作圖



```
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X_flat)

# 降到 50 維
pca_50 = PCA(n_components=50, random_state=42)
X_pca50 = pca_50.fit_transform(X_scaled)

# 降到 2 維
pca_2 = PCA(n_components=2, random_state=42)
X_pca2 = pca_2.fit_transform(X_scaled)

print("PCA50:", X_pca50.shape, "PCA2:", X_pca2.shape)
```

PCA50: (30000, 50) PCA2: (30000, 2)

```
K = 5 # 我分5群
kmeans = KMeans(n_clusters=K, random_state=42, n_init=10)
clusters = kmeans.fit_predict(X_pca50)

print("各群大小:", np.bincount(clusters))

plt.figure(figsize=(6, 5))
for k in range(K):
    idx = (clusters == k)
    plt.scatter(X_pca2[idx, 0], X_pca2[idx, 1], s=3, alpha=0.5, label=k)
plt.legend(markerscale=3)
plt.xlabel("PC1")
plt.ylabel("PC2")
plt.title("PCA 2D + K-means Clusters")
plt.show()
```

微結構聚類與識別

PCA + K-means Clustering

Coarse Second-Phase / High-Contrast

Mixed Boundary + Medium Contrast Transitional Zones

Grain-Boundary-Dominant with Light Second-Phase Presence

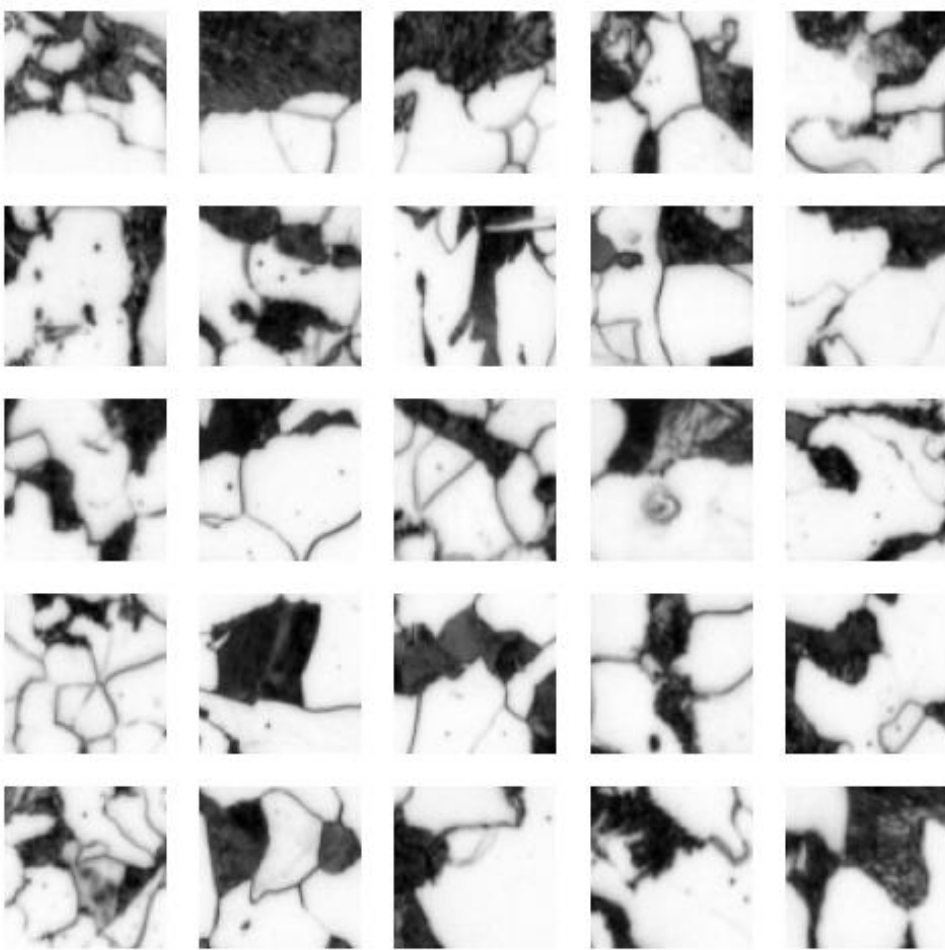
Cluster 0 Representative Patches



Cluster 1 Representative Patches

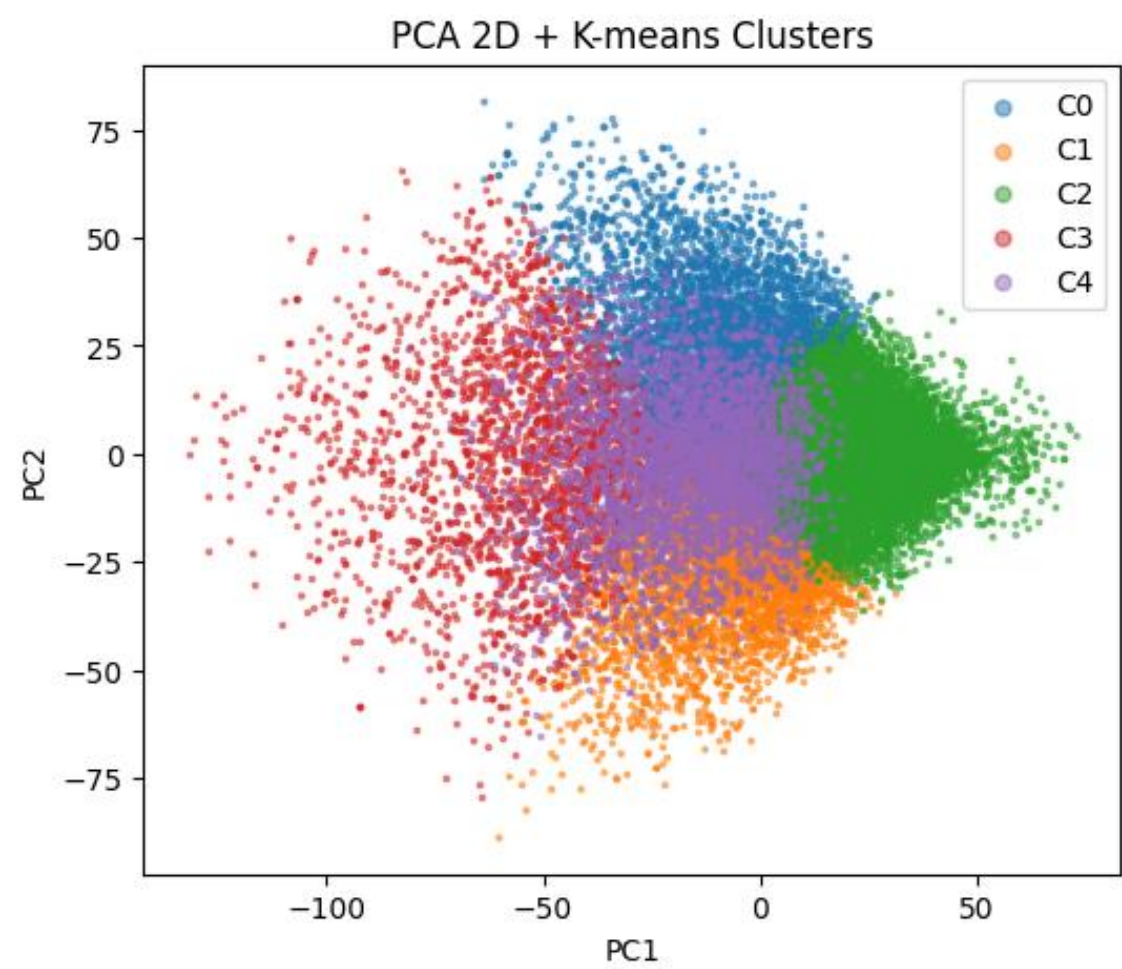


Cluster 2 Representative Patches



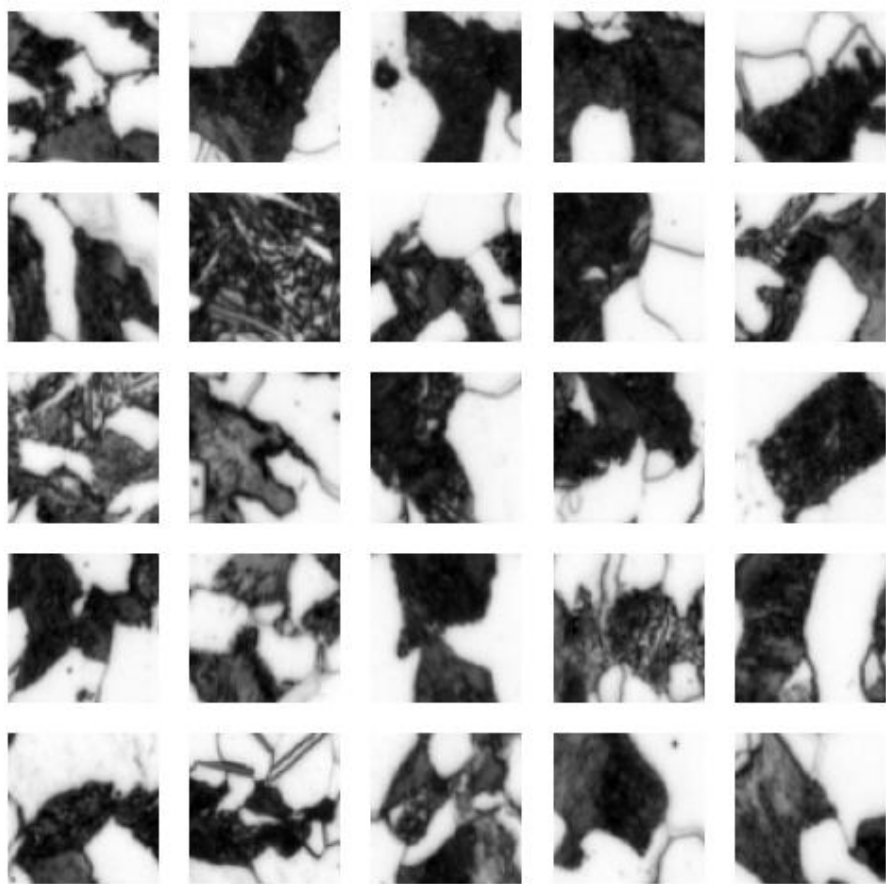
微結構聚類與識別

PCA + K-means Clustering



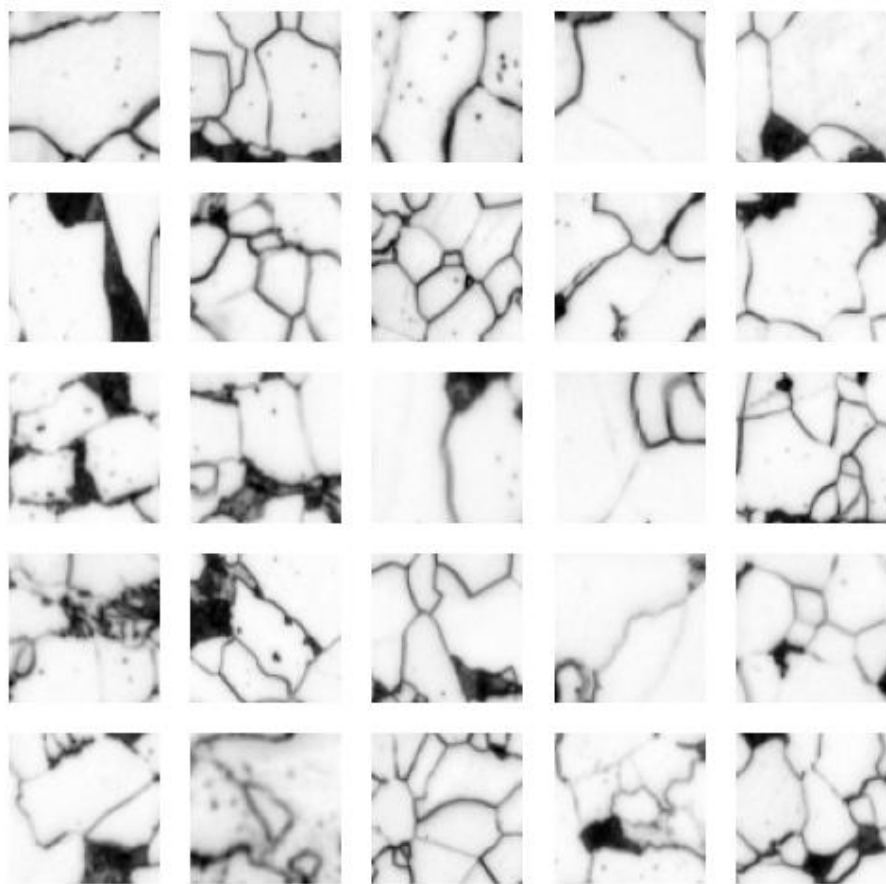
Dense Dark-Phase / Highly Textured Zones

Cluster 3 Representative Patches



Grain Boundary / Low-Texture Regions

Cluster 4 Representative Patches





Convolutional AutoEncoder (CAE) 架構

Encoder

壓縮：學習 SEM 的核心特徵（像是 grain boundary、明暗紋理、孔洞等）。
壓到一個很小的 feature vector（例如 16 維或 32 維的 latent space）。

Decoder

還原：嘗試還原 patch
把壓縮後的特徵解碼、重建回一張 64x64 的 SEM 影像

缺陷偵測原理

正常 microstructure → 容易重建 → error 低
缺陷 microstructure → 重建失敗 → error 高

*為什麼 CAE 適合 SEM?

缺陷檢測原理：Reconstruction Error Heatmap

檢測缺陷的核心是透過比較「原始輸入圖像」與「模型重建後圖像」之間的差異來識別缺陷

工作機制

- 模型在重建缺陷（如劃痕、空洞）時會失敗，導致重建圖像與原始圖像存在顯著差異。
- 透過計算像素級差異，我們可以生成一個殘差圖。
- 殘差圖中較高的像素差異值，即代表潛在的缺陷位置。

```
from sklearn.model_selection import train_test_split
```

```
X_all = patches.astype(np.float32)
```

```
MAX_TRAIN = 20000 #60000改20000(加快速度)
```

```
N_train_use = min(X_all.shape[0], MAX_TRAIN)
```

```
X_sub = X_all[:N_train_use]
```

```
X_train, X_val = train_test_split(X_sub, test_size=0.1, random_state=42)
print("Train:", X_train.shape, " Val:", X_val.shape)
```

```
Train: (18000, 64, 64, 1) Val: (2000, 64, 64, 1)
```

```
import tensorflow as tf
```

```
from tensorflow.keras import layers, models
```

```
input_shape = (64, 64, 1)
```

```
inp = layers.Input(shape=input_shape)
```

```
x = layers.Conv2D(32, (3, 3), padding='same', activation='relu')(inp)
```

```
x = layers.MaxPool2D((2, 2))(x) # 32x32
```

```
x = layers.Conv2D(64, (3, 3), padding='same', activation='relu')(x)
```

```
x = layers.MaxPool2D((2, 2))(x) # 16x16
```

```
x = layers.Conv2D(128, (3, 3), padding='same', activation='relu')(x)
```

```
encoded = layers.MaxPool2D((2, 2))(x) # 8x8x128
```

```
x = layers.Conv2DTranspose(128, (3, 3), strides=2, padding='same', activation='relu')(encoded)
```

```
x = layers.Conv2DTranspose(64, (3, 3), strides=2, padding='same', activation='relu')(x)
```

```
x = layers.Conv2DTranspose(32, (3, 3), strides=2, padding='same', activation='relu')(x)
```

```
decoded = layers.Conv2D(1, (3, 3), padding='same', activation='sigmoid')(x)
```

```
autoencoder = models.Model(inp, decoded)
```

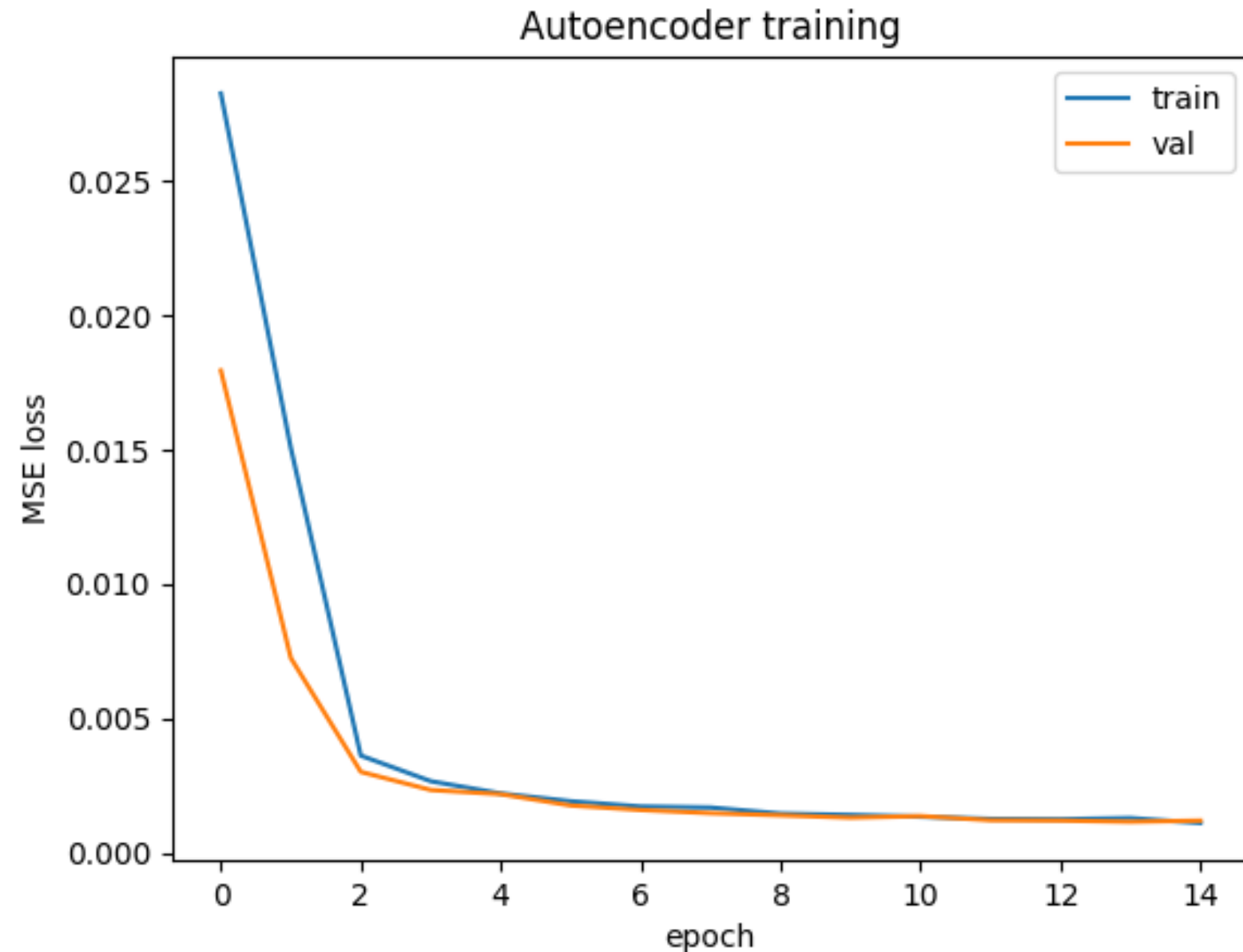
```
autoencoder.compile(optimizer=tf.keras.optimizers.Adam(1e-3), loss='mse')
```

```
autoencoder.summary()
```


缺陷检测原理：Reconstruction Error Heatmap

```
callbacks = [  
    tf.keras.callbacks.ReduceLROnPlateau(  
        monitor='val_loss', factor=0.5, patience=5, min_lr=1e-5  
    ),  
    tf.keras.callbacks.EarlyStopping(  
        monitor='val_loss', patience=10, restore_best_weights=True  
    )  
]  
  
history = autoencoder.fit(  
    X_train, X_train,  
    validation_data=(X_val, X_val),  
    epochs=15, #100改15  
    batch_size=256,  
    shuffle=True,  
    callbacks=callbacks  
)
```

```
import matplotlib.pyplot as plt  
plt.plot(history.history['loss'], label='train')  
plt.plot(history.history['val_loss'], label='val')  
plt.xlabel('epoch')  
plt.ylabel('MSE loss')  
plt.legend()  
plt.title('Autoencoder training')  
plt.show()
```



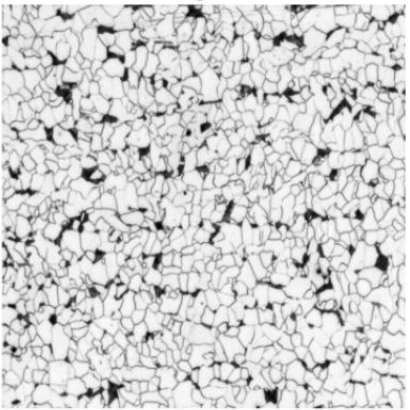
實驗結果：可視化缺陷

以下是缺陷检测过程的视觉化展示，清晰呈现了从原始图像到最终缺陷定位的步骤。

1

原始 SEM 图像

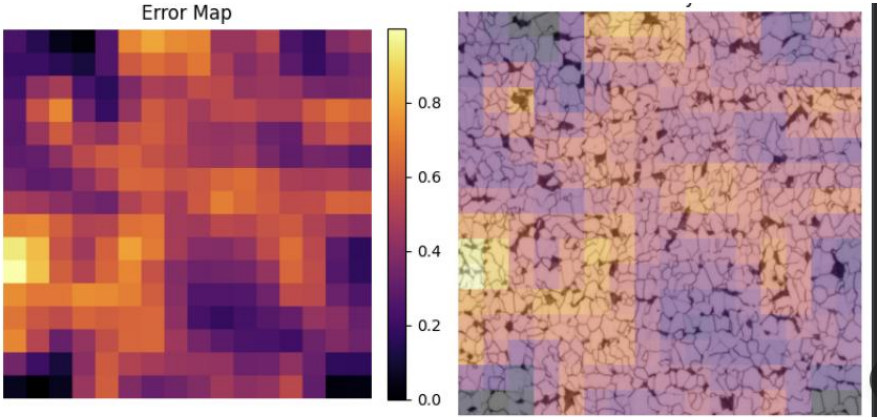
显示带有缺陷的微结构



2

重建误差热力图

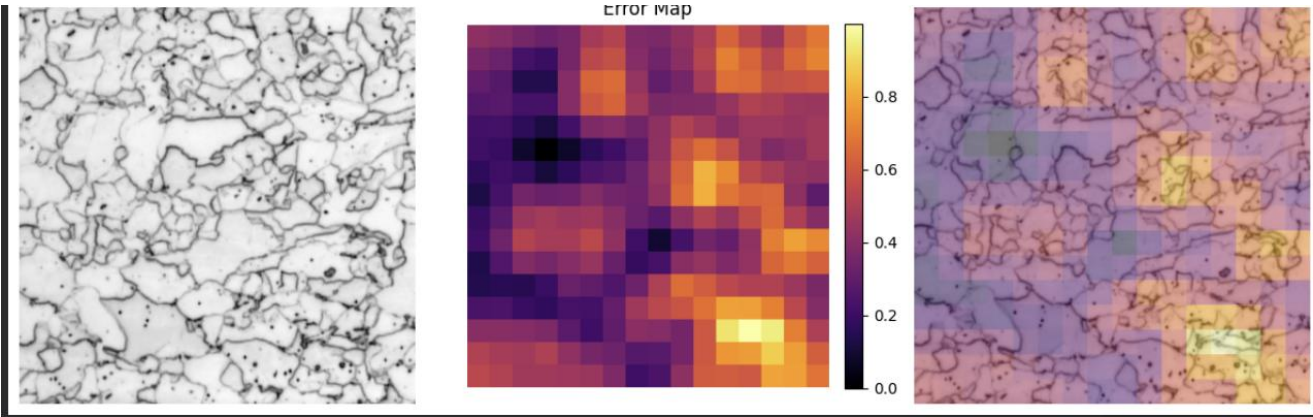
突出显示高误差区域，
即潜在缺陷



3

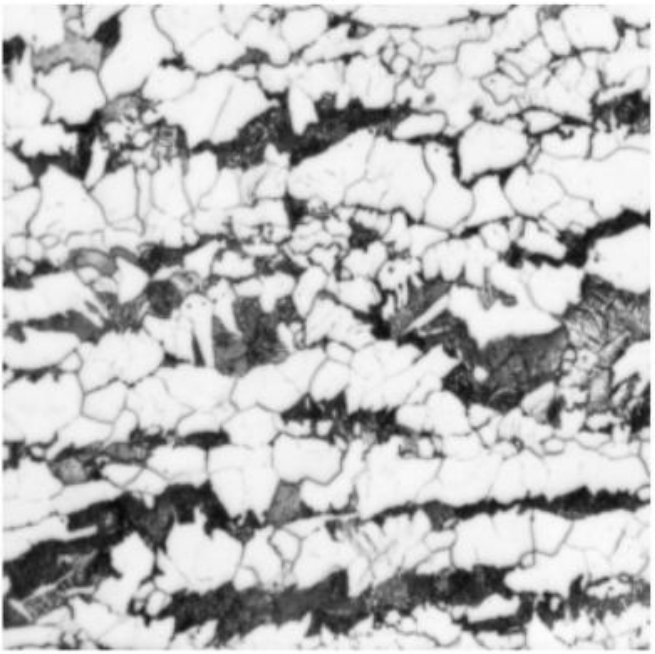
最终二值化遮罩

隔离并标识出缺陷位置



误差最高5%的 pixel：视为缺陷（1，白色）
其余95%：正常（0，黑色）

Original



Defect Mask (top 5% error)





結論

- 成功建立一套無監督 SEM 微結構分析管線，可處理大量材料影像並自動萃取特徵。
- K-means 分群結果具材料科學上的一致性，能反映不同微結構形態。
- Autoencoder 的重建誤差可生成缺陷熱圖與 defect mask，準確定位異常區域。
- 方法不依賴人工標註，具高擴展性，適合應用於品質檢測與製程監控。

參考文獻：

- [1] D. Kumar, P. C. Agraharam, and S. Namilae, "Unsupervised anomaly detection in composite manufacturing using autoencoders and cluster-specific thresholding," *Manuf. Lett.*, vol. 45, pp. 101-106, 2025, doi: 10.1016/j.mfglet.2025.08.001.
- [2] C. Kunselman, S. Sheikh, M. Mikkelsen, V. Attari, and R. Arróyave, "Microstructure classification in the unsupervised context," *Acta Mater.*, vol. 223, Art. no. 117434, 2022, doi: 10.1016/j.actamat.2021.117434.